

Interview story: FDA behavior-clone system ("parallel")

Use for: "Tell me about a challenging project," "a system you designed end to end," "a time you used AI/LLMs in production," "dealing with ambiguity."

The 2-minute version (STAR)

Situation. I wanted a system that tracks changes in the FDA regulatory world — new guidances, warning letters, recalls — and predicts how each stakeholder behaves in response: what enforcement does next, what reviewers will ask for, how industry adapts. News monitoring tells you what happened; nothing told you what happens next.

Task. Build it end to end on parallel.ai's API suite: sense changes, extract structured understanding, build behavior-pattern "clones" of the three stakeholders, simulate their responses side by side, and surface it on a live dashboard.

Action. - Designed a five-stage pipeline: **Sense** (Search / Extract / Monitor) → **Understand** (Task API with JSON-schema output contracts) → **Clone** (FindAll + Task for pattern mining, Chat API for the clone runtime) → **Simulate** (fault-tolerant fan-out to all clones) → **Surface** (FastAPI dashboard, Railway, auto-deploy from GitHub). - The core design decision: clones are not just personas in a system prompt. I built a **pattern library** — distilled behavior patterns such as "501(j) inspection refusal accelerates enforcement toward import alert," each with trigger phrases, evidence citations, and confidence. A deterministic matcher scores every pattern per stakeholder (trigger coverage × confidence, no LLM call), injects the matches into the clone's prompt as its evidence base, and every prediction cites which patterns fired. Explainable, not vibes. - Built for reality: the full pipeline runs in demo mode with zero API key, seeded from 8 real FDA publications (real titles/URLs/dates, synthetic analyses labeled as demo), so I could develop and demo the entire loop before the key arrived.

Result. Live dashboard: filterable event feed, per-event structured analysis, and side-by-

side predictions from all three clones with pattern citations and confidence. Clean separation of concerns — mining (batch, expensive) vs. matching (deterministic, free) vs. reasoning (LLM) — means adding a fourth stakeholder is a new profile plus new patterns; nothing else changes.

Follow-up questions I can answer

- **Why deterministic matching instead of embedding retrieval?** Explainability and cost. Every fired pattern shows its score, matched triggers, and confidence; no hallucinated evidence. Embeddings would add infra for marginal gain at this scale.
- **How would you evaluate the clones?** Backtest: run historical changes through the clones, compare predicted vs. observed outcomes, measure directional accuracy and calibration of confidence scores.
- **Hardest bug?** Pattern injection had to be stakeholder-scoped — an early version let enforcement patterns leak into the reviewer clone's prompt and the behaviors cross-contaminated. Fixed by partitioning the library per stakeholder in the matcher.
- **What would you do with more time?** Live mining loop on a schedule, calibration tracking per pattern (promote/demote confidence by observed accuracy), and a dissent view highlighting where clones disagree.
- **Ambiguity?** parallel.ai's docs left several API fields uncertain (monitor payload shape, citation fields, task output shapes). I isolated every uncertainty behind a thin client with defensive parsing and TODO markers, so verification with a real key touches one file.